

Memoria Técnica de Ingeniería del Software

Sistema de Gestión de Soporte Técnico: mgcss-track

Módulo L2 - Grupo G8

**Perspectiva Formal sobre Decisiones de Diseño, Métricas de Calidad y
Gestión de Deuda Técnica**

4 de junio de 2026

Índice

1. Introducción y Propósito del Sistema	2
2. Decisiones Arquitectónicas y de Diseño	2
2.1. Separación de Conceptos (Layers)	2
2.2. Patrones de Diseño Aplicados	2
3. Instrucciones de Instalación y Despliegue	2
3.1. Requisitos Previos	3
3.2. Opción 1: Ejecución Local en Desarrollo	3
3.3. Opción 2: Despliegue mediante Docker	3
4. Comparativa de Métricas de Calidad	3
5. Análisis de Deuda Técnico	3
5.1. Falta de Persistencia del Historial de Estados (Deuda de Infraestructura)	4
5.2. Uso de Tipos de Datos Heredados (Legacy Date API)	4

1 Introducción y Propósito del Sistema

El proyecto `mgcss-track` consiste en el diseño e implementación de una API REST formal destinada a gestionar el ciclo de vida de las solicitudes de soporte técnico. El sistema permite modelar estados transicionales de negocio rigurosos (`ABIERTA` → `PROCESANDO` → `CERRADA`), garantizando restricciones de integridad como la imposibilidad de asignar técnicos inactivos o realizar mutaciones sobre entidades en estados terminales, salvo por la operación explícita de reapertura.

El desarrollo se ha guiado bajo una metodología estricta de Desarrollo Guiado por Pruebas (TDD), persiguiendo una separación de conceptos limpia y un pipeline de Integración Continua (CI) automatizado que actúe como un guardián de la calidad del software.

2 Decisiones Arquitectónicas y de Diseño

El sistema se ha estructurado siguiendo los principios de la Arquitectura en Capas Limpia, garantizando el desacoplamiento de la infraestructura frente a las reglas esenciales de negocio.

2.1 Separación de Conceptos (Layers)

La estructura de paquetes implementada refleja fielmente el aislamiento de responsabilidades:

- **domain/:** Núcleo puro del sistema. Contiene las entidades esenciales (`Solicitud`, `Tecnico`), los Value Objects (`RegistroEstado`) y los enumerados transicionales. No posee ninguna dependencia hacia frameworks externos. Las reglas de validación residen exclusivamente aquí.
- **service/:** Capa de Casos de Uso. Orquesta la ejecución de la lógica del dominio, interactuando con las interfaces de abstracción de persistencia. Carece de lógica condicional compleja de negocio.
- **infraestructura/:** Capa de Adaptadores e Infraestructura. Contiene las entidades JPA moleculares (`SolicitudEntity`, `TecnicoEntity`), los repositorios de Spring Data y la implementación del patrón *Adapter* (`JpaSolicitudRepositoryAdapter`), aislando la base de datos (H2 en memoria).
- **controller/:** Capa de Presentación/Exposición. Define los controladores REST y los Objetos de Transferencia de Datos (DTO) para desacoplar el contrato externo de la API de las estructuras internas.

2.2 Patrones de Diseño Aplicados

- **Patrón Repository / Adapter:** Se introducen interfaces en la infraestructura que exponen dominio puro, abstrayendo el framework ORM. El adaptador se encarga de realizar el mapeo bidireccional entre el modelo rico del dominio y el modelo anémico de persistencia.
- **Value Objects para el Historial de Estados:** La clase `RegistroEstado` se concibe como un objeto sin identidad propia, mutable solo por sustitución, que documenta de forma cronológica el ciclo de vida transicional de una `Solicitud`.

3 Instrucciones de Instalación y Despliegue

El proceso de construcción y ejecución ha sido completamente verificado a través de los artefactos generados en la release automatizada del repositorio.

3.1 Requisitos Previos

- Java Development Kit (JDK) 17.
- Docker Engine (opcional, para entornos contenedorizados).

3.2 Opción 1: Ejecución Local en Desarrollo

Para compilar y arrancar la aplicación utilizando el Maven Wrapper incluido en el submódulo `l2g8`:

```
cd l2g8
./mvnw clean spring-boot:run
```

La API estará expuesta en <http://localhost:8080>. La interfaz interactiva de Swagger UI se puede acceder en <http://localhost:8080/swagger-ui/index.html>.

3.3 Opción 2: Despliegue mediante Docker

El proyecto cuenta con un archivo `Dockerfile` optimizado sobre la base de `eclipse-temurin:17-jre-alpin`.

```
cd l2g8
./mvnw clean package -DskipTests
docker build -t mgcss-track .
docker run -p 8080:8080 -e APP_PORT=8080 --name mgcss-track-container
mgcss-track
```

4 Comparativa de Métricas de Calidad

A continuación, se presenta un análisis comparativo de la evolución interna del código tras las últimas sesiones de refactorización y aseguramiento de la calidad, en consonancia con las directrices de SonarCloud.

Tabla 1: Evolución de Métricas del Sistema

Métrica		Estado Inicial	Estado Refactorizado (Actual)
Complejidad Ciclomática		Elevada (Múltiples <code>if</code>)	Reducida (Extract Method)
Code Smells (JUnit 5)		Advertencias (<code>public</code> redundantes)	0 (Limpieza absoluta)
Cobertura (JaCoCo)		< 80 %	≥ 80 % (Garantizado en Pipeline)
Maintainability (Sonar)	Rating	Grado B/C	Grado A
Bugs & Vulnerabilidades		Potenciales por acoplamiento	0

El Quality Gate del Pipeline CI exige estrictamente un 0 % de fallos catastróficos y un umbral mínimo del 80 % de cobertura para autorizar la integración en la rama principal (`main`).

5 Análisis de Deuda Técnica

A pesar de la obtención de la calificación **A** en mantenibilidad, se han identificado focos de deuda técnica controlada que deben ser gestionados en las iteraciones subsiguientes.

5.1 Falta de Persistencia del Historial de Estados (Deuda de Infraestructura)

- **Descripción:** El historial de transiciones de estado mapeado en la lista `historialEstados` de la entidad de dominio `Solicitud` se gestiona únicamente en memoria durante la vida transaccional. El adaptador de base de datos (`SolicitudEntity`) no almacena estos registros en tablas relacionales.
- **Justificación Temporal:** Introducir de forma inmediata el mapeo relacional mediante `@ElementCollection` durante una fase de parche crítico incrementaba de manera inaceptable el riesgo de regresión en el esquema de la base de datos.
- **Estrategia de Mitigación:** Se ha registrado un Issue técnico formal para la siguiente iteración, donde se implementará el mapeo persistente mediante JPA, asegurando la coherencia a largo plazo sin alterar la estabilidad del dominio.

5.2 Uso de Tipos de Datos Heredados (Legacy Date API)

- **Descripción:** El dominio expone la clase clásica `java.util.Date`, obligando a la capa de infraestructura a realizar conversiones explícitas hacia `java.time.LocalDateTime`.
- **Impacto:** Introduce transformaciones de tipos propensas a sutiles errores latentes de zona horaria (*Timezone*).
- **Estrategia de Mitigación:** Refactorizar el tipo primitivo en la entidad raíz de dominio hacia `java.time.Instant` o `LocalDateTime` nativo en la próxima ventana de refactorización estructural.